

Block 1: Grundlagen & Orientierung

Workshop: Agentic Engineering

Thomas Hein — Datacidars

Blick in die Zukunft — Was bleibt fuer uns uebrig?

"Wer von euch findet gut, was gerade in der Softwareentwicklung passiert?"

Die grosse Debatte

"Nobody has to program. The programming language is human."
— Jensen Huang, 2024

"This is the best time yet to learn to code."
— Bill Gates, 2025

"You are orchestrating agents who do and acting as oversight."
— Andrej Karpathy, 2026

Zahlen und Fakten

Unternehmen	AI-generierter Code	Quelle
Google	>30%	Pichai, Q1 2025
Microsoft	20-30%	Nadella, Apr 2025
Meta	Ziel: 50% bis 2026	Zuckerberg, 2025

- **Gartner:** 90% der Enterprise-Entwickler nutzen AI bis 2028
- **Aber:** 2500% mehr Defekte durch unkontrollierte AI-Nutzung

Kernthese des Workshops

1. Softwareentwicklung wird **niederschwelliger**
2. Es wird relevanter, Agenten zu **beherrschen** und **praezise auszudruecken** was gefordert ist
3. Die Rolle verschiebt sich: **Code-Schreiber** → **Architekt, Steuermann, Qualitaetssicherer**
4. **Der Entwickler wird zum Boss des Agenten, nicht umgekehrt**

Quellen & Weiterlesen — Blick in die Zukunft

- [Jensen Huang: "The programming language is human" \(Tom's Hardware, 2024\)](#) — NVIDIA CEO ueber AI und Programmierung
- [Bill Gates: "Best time to learn to code" \(Windows Central, 2025\)](#) — Gegenperspektive: AI als Werkzeug, nicht Ersatz
- [Andrej Karpathy: Agentic Engineering \(X/Twitter, 2026\)](#) — Karpathys Vision: Orchestrierung statt direktes Coden
- [Gartner: 90% of enterprise developers will use AI by 2028 \(April 2024\)](#) — Marktprognose mit Warnung vor unkontrollierter Nutzung

Das 4-Stufen-Modell

#	Ansatz	Input	Menschliche Rolle
1	Vibe Coding	Kurze Phrasen, fachliche Wuensche	Fachanwender
2	Spec-Driven (Business)	Pflichtenheft, Use Cases	IT Consultant
3	Spec-Driven (Technical)	UML, ERM, Architektur	Architekt
4	Agentic Engineering	Anforderungen + kollaborative Umsetzung	Architekt/Entwickler

Wann welche Stufe?

- **Vibe Coding:** Prototypen, Demos, Wegwerf-Experimente
- **Spec-Driven (Business):** Wenn Fachbereich fuehrt, wenig technische Tiefe noetig
- **Spec-Driven (Technical):** Neue Systeme, klare Architektur, gruene Wiese
- **Agentic Engineering:** Produktionscode, bestehende Systeme, Qualitaet entscheidend

Kleine UI-Aenderung ≠ neues Modul — die Methodik muss zur Aufgabe passen.

Quellen & Weiterlesen — Das 4-Stufen-Modell

- [Andrej Karpathy: "Vibe Coding" \(X/Twitter, Feb 2025\)](#) — Ursprung des Begriffs "Vibe Coding"
- [Andrej Karpathy: Software is Changing \(X/Twitter, Feb 2026\)](#) — Karpathys Begriff "Agentic Engineering"
- [Thoughtworks: "Preparing your team for the agentic software development life cycle"](#) — Spec-Driven Ansätze im Vergleich
- [DEV Community: "The AI Coding Workflow That Actually Works: Separate Planning from Execution"](#) — Praxis-Perspektive auf Agenten-Ansätze

Wie funktioniert ein Coding Agent?

LLM + Tools + Kontext = Agent

Der Kreislauf:

1. **Prompt** empfangen
2. **Denken** (Reasoning)
3. **Tool aufrufen** (Dateien lesen, Code schreiben, Tests ausführen)
4. **Ergebnis** auswerten
5. **Weiter denken** oder antworten

[BILD: Agent Loop Diagramm — Quelle: <https://dev.to/jakesweb/forget-the-hype-agents-are-loops-2fi5>]

Autocomplete → Chat → Agent

Modus	Wie es funktioniert	Beispiel
Autocomplete	Inline-Vorschlaege beim Tippen	Tab-Completion in IDE
Chat	Frage-Antwort im Seitenpanel	"Erklaere diese Funktion"
Agent	Autonome Ausfuehrung mit Tool-Zugriff	"Implementiere Feature X mit Tests"

Das Oekosystem eines Coding Agents

- **Skills/Rules:** Vordefinierte Workflows und Regeln (z.B. TDD-Skill)
- **Plugins:** Erweiterungen (z.B. Context7 fuer aktuelle Doku)
- **MCP-Server:** Externe Datenquellen anbinden (DB, Jira, Git)
- **Hooks:** Automatische Aktionen bei bestimmten Events

[BILD: MCP Host-Client-Server Architektur — Quelle:

<https://modelcontextprotocol.io/docs/concepts/architecture>]

Quellen & Weiterlesen — Funktionsweise Coding Agent

- [DEV Community: "Forget the Hype: Agents are Loops"](#) — Praxis-Erklärung des Agent-Loop-Konzepts
- [Oracle: "What Is the AI Agent Loop?"](#) — Perceive/Reason/Plan/Act/Observe Schleife
- [Model Context Protocol — Architektur](#) — MCP Host-Client-Server Konzept
- [GitHub Blog: Copilot Ask, Edit, Agent modes](#) — Drei Modi im Vergleich

Wie aendert sich unsere Arbeitsweise?

Vom Code-Tipper zum Architekten und Steuermann

- Du schreibst weniger Code — du **steuerst** mehr
- Pair-Programming mit dem Agenten: Zu zweit den Agenten steuern
- Fokus verschiebt sich: Technische Qualitaet, Fachlichkeit, Konzeptionsfaehigkeit

Hypothese: Was bedeutet das fuer Teams?

- Teams werden **kleiner** — max. 2 Entwickler pro Product Owner
- **Anforderungsaufnahme** wird **laenger** als die Entwicklung
- Der Product Owner wird zum Engpass, nicht der Entwickler
- Qualitaet der Anforderungen bestimmt Qualitaet des Ergebnisses

Quellen & Weiterlesen — Veraenderte Arbeitsweise

- [Andrej Karpathy: "You are orchestrating agents" \(X/Twitter, 2026\)](#) — Die neue Rolle: Steuermann statt Tipper
- [Thoughtworks: Preparing your team for the agentic SDLC](#) — Teamstruktur und Rollen im agentic Umfeld
- [GitHub Blog: Quantifying GitHub Copilot's impact on developer productivity](#) — Entwickler 55,8% schneller mit AI-Unterstützung
- [InfoQ: From Prompts to Production — a Playbook for Agentic Development](#) — Praxis-Playbook fuer Teams

Demo: Gleiche Aufgabe, zwei Ansaetze

Aufgabe: *[ENTSCHEIDUNGSPUNKT: Thomas waehlt Demo-Aufgabe vor Workshop]*

	Vibe Coding	Agentic Engineering
Input	"Bau mir X"	Spec → Plan → TDD
Designentscheidungen	Agent entscheidet	Entwickler entscheidet
Tests	Vielleicht	Erst Test, dann Code
Nachvollziehbarkeit	Gering	Hoch (Plan, Commits, Doku)

Quellen & Weiterlesen — Demo und Ansätze im Vergleich

- [Andrej Karpathy: Vibe Coding \(X/Twitter, Feb 2025\)](#) — Ursprung des Begriffs: Coding im Flow ohne Code-Verständnis
- [The New Stack: Vibe Coding is Passé](#) — Warum Vibe Coding an Grenzen stößt und Agentic Engineering folgt
- [Dev.to: Separate Planning from Execution — The AI Coding Workflow That Actually Works](#) — Praxis-Vergleich beider Ansätze mit konkretem Workflow
- [GitHub Blog: Test-Driven Development with GitHub Copilot](#) — TDD als Grundlage für nachvollziehbares Agentic Engineering
- [Glide Blog: What is Agentic Engineering](#) — Karpathys Definition und Abgrenzung zu Vibe Coding